

Khởi tạo (Initialization)

PHỤ THUỘC A/01-hình-hoc-da-thi (E so với H , bốn cấu hình suy biến — chương này là ứng dụng trực tiếp nhất của mục 6 ở đó); A/05-observability-gauge-va-suy-bien (điều kiện kích thích, và nguyên tắc từ chối khi chưa đủ); B/07-dac-trung-va-data-association (tương ứng đầu vào); B/09-paradigm-uoc-luong-back-end (mỗi kiến trúc có yêu cầu khởi tạo khác nhau).

TÓM TẮT

Mọi bộ tối ưu phi tuyến cần một điểm khởi đầu đủ gần, và SLAM bắt đầu từ chỗ không biết gì. Chương này về khoảnh khắc chuyển từ “không biết gì” sang “có một ước lượng đủ tốt để Gauss-Newton hội tụ”. Nội dung có ba phần. Một: khởi tạo mono hai khung — bài toán khó nhất — với kỹ thuật model selection chạy song song E và H rồi để dữ liệu quyết định; đây là chỗ bốn cấu hình suy biến ở A/01 chuyển từ lý thuyết thành mã. Hai: các trường hợp dễ hơn — stereo, RGB-D, thị giác-quán tính, prior học được — và điều đáng chú ý là mỗi trường hợp dễ hơn *vì cùng một lý do*: có thêm một nguồn thông tin phá vỡ một gauge. Ba: phần quan trọng nhất và bị bỏ qua nhiều nhất, là *tiêu chí từ chối* và cơ chế phát hiện khởi tạo hỏng. Nếu chỉ nhớ một điều: *một bộ khởi tạo tốt không phải bộ luôn trả về kết quả mà là bộ biết từ chối khi dữ liệu chưa đủ* — vì một khởi tạo sai không tự sửa mà nhiễm độc toàn bộ bản đồ về sau.

KIẾN THỨC CẦN CÓ TRƯỚC

- **khởi tạo (initialization)** — quá trình tạo ra ước lượng đầu tiên của tư thế và cấu trúc từ chỗ không có tiên nghiệm nào. Khác với tracking ở chỗ tracking luôn có một dự đoán từ khung trước.
- **model selection** — chạy song song nhiều mô hình hình học (E và H), tính điểm số cho mỗi mô hình dựa trên phần dư của inlier, rồi chọn mô hình thắng. Cần thiết vì hai mô hình suy biến ở những chỗ bù nhau.
- **parallax** — góc giữa hai tia nhìn tới cùng một điểm từ hai vị trí camera. Đây là đại lượng quyết định độ sâu có xác định được không, và là tiêu chí đúng để đánh giá một cặp khung khởi tạo.
- **bootstrapping** — giai đoạn trước khi khởi tạo thành công, trong đó hệ thống thu thập khung và theo dõi đặc trưng nhưng chưa có bản đồ 3D nào.
- **khởi tạo thị giác-quán tính** — quá trình giải cho hướng trọng lực, tỉ lệ metric, vận tốc ban đầu và độ lệch con quay bằng cách căn chỉnh quỹ đạo thị giác (không có tỉ lệ) với tiền tích phân IMU. Chi tiết ở ../IMU_Fusion/ chương 10.
- **khởi tạo lại (re-initialization)** — quá trình bắt đầu lại sau khi hệ thống mất bám hoàn toàn. Khác với relocalization ở chỗ relocalization tìm lại vị trí *trong bản đồ cũ*, còn khởi tạo lại tạo một bản đồ mới.

CÂU HỎI MỞ ĐẦU

- Q1. Bộ khởi tạo của bạn thành công 100% số lần được gọi — nó chưa bao giờ từ chối. Vì sao đó là một tin xấu chứ không phải tin tốt?
- Q2. ORB-SLAM chạy song song hai mô hình E và H rồi chọn. Vì sao không đơn giản là luôn dùng thuật toán năm điểm, vốn không suy biến với cảnh phẳng?

- Q3. Khởi tạo stereo gần như tầm thường: có độ sâu ngay từ khung đầu. Nêu chính xác điều gì stereo cung cấp mà mono không có, và nói điều đó liên hệ thế nào với A/05.
- Q4. Một khởi tạo sai với tỉ lệ lệch 20% nhưng hình học tự nhất quán. Bundle adjustment chạy sau đó sẽ sửa được không, và vì sao?
- Q5. Hệ thống của bạn khởi tạo tốt trong phòng thí nghiệm và thất bại ở hành lang khách hàng. Nêu ba nguyên nhân hình học khác nhau, và nêu đại lượng phân biệt chúng.

1.1 Đặt vấn đề

A/04 dựng một bộ giải mạnh và nêu một điều kiện mà nó không tự thoả được: Gauss-Newton cần một điểm khởi đầu đủ gần nghiệm. Với bài toán không lồi như BA, “đủ gần” không phải một hình thức — xa quá thì bộ giải hội tụ về một cực tiểu địa phương hoàn toàn sai, và nó làm điều đó một cách hoàn toàn tự tin.

Bài toán gốc: hệ thống vừa được bật lên. Nó có một chuỗi ảnh và không có gì khác — không tư thế, không bản đồ, không biết mình đang ở đâu hay cảnh trông thế nào. Từ chỗ đó, phải tạo ra một ước lượng đủ tốt.

Điều làm chương này quan trọng hơn về ngoài là *hậu quả của việc làm sai không tự giới hạn*. Một khung tracking sai sẽ được sửa ở khung sau. Một khởi tạo sai thì không: bản đồ ban đầu trở thành hệ quy chiếu cho mọi thứ sau đó, và một bản đồ khởi tạo méo sẽ làm mọi landmark thêm vào sau bị méo theo cho khớp với nó. Hệ thống hội tụ, phần dư nhỏ, và toàn bộ bản đồ sai. Đây là câu trả lời cho câu hỏi mở đầu thứ tư: BA *không* sửa được, vì BA tìm cầu hình nhất quán nhất với dữ liệu, và một bản đồ méo với tỉ lệ sai là nhất quán — đó chính là gauge tỉ lệ ở A/05 mục 3.

Ai gặp bài toán này? Mọi hệ SLAM, và nó gay gắt nhất với mono. Với stereo hay RGB-D thì gần như tầm thường, và *vì sao* nó tầm thường là một bài học đáng rút ra chứ không phải một may mắn.

Trước khi có model selection, cách làm phổ biến là chọn sẵn một mô hình — thường là E — và chờ tới khi nó cho kết quả trông hợp lý. Cách đó hỏng trong nhà, nơi cảnh phẳng là quy luật chứ không phải ngoại lệ: camera nhìn vào tường, sàn, trần, cửa. Với cảnh phẳng, E không xác định duy nhất (A/01 mục 6B), và nghiệm trả về là một tổ hợp tùy tiện do nhiễu quyết định — nhưng số inlier vẫn cao, nên “trông hợp lý”.

Đó là lý do sâu xa để chạy hai mô hình. Và có một lợi ích thứ hai ít được nhắc nhưng quan trọng hơn cho sản phẩm: khi H thắng, hệ thống *biết* nó đang nhìn một mặt phẳng hoặc đang xoay thuần, và nó có thể đổi *hành vi* — chờ thêm, báo cho người dùng di chuyển — chứ không chỉ đổi công thức.

TRỰC GIÁC

Hình dung bạn tỉnh dậy trong một căn phòng lạ và phải vẽ bản đồ của nó chỉ bằng cách nhìn.

Nhìn một lần không cho biết gì về chiều sâu — mọi thứ có thể là một bức tranh tường phẳng. Bạn phải *đi vài bước* rồi nhìn lại, và so sánh: thứ gì dịch nhiều thì gần, thứ gì gần như đứng im thì xa.

Ba chuyện có thể làm hỏng việc này, và biết ba chuyện đó là biết cả chương.

Bạn xoay đầu thay vì bước đi. Cảnh đổi hoàn toàn, nhưng không có gì dịch tương đối với nhau, nên bạn không học được gì về chiều sâu. Tệ hơn, bạn có cảm giác đã quan sát rất nhiều.

Bạn đang nhìn vào một bức tường phẳng. Bạn bước đi, mọi thứ dịch — nhưng chúng dịch giống hệt nhau, vì chúng ở cùng khoảng cách. Bạn không đọc được gì từ sự khác biệt, vì không có khác biệt nào.

Bạn đi thẳng về phía thứ mình đang nhìn. Nó to dần lên nhưng không lệch sang bên. Bạn có một chút thông tin, rất ít, và ước lượng của bạn chậm chạp.

Trong cả ba trường hợp, một người khôn ngoan sẽ nói “khoan đã, tôi chưa đủ dữ liệu” rồi bước sang bên vài bước. Một cái máy được lập trình kém sẽ vẽ ra một bản đồ, vì người ta bảo nó phải vẽ ra một bản đồ.

Toàn bộ chương này là về việc dạy cái máy nói câu “khoan đã”.

1.2 Khởi tạo mono: bài toán khó nhất

A. Quy trình

Bốn bước, và bước thứ tư mới là bước phân biệt một bộ khởi tạo tốt với một bộ tồi.

Bước một — thu thập và theo dõi. Chọn một khung tham chiếu, theo dõi đặc trưng qua các khung sau (B/07). Tích lũy cho tới khi có đủ tương ứng và đủ parallax.

Bước hai — chạy song song hai mô hình. Với cùng tập tương ứng, chạy RANSAC cho E (hoặc F) và cho H . Mỗi mô hình cho một tập inlier và một tổng phần dư.

Bước ba — chọn mô hình. Tính một điểm số cho mỗi mô hình phản ánh cả số inlier lẫn độ khớp, rồi chọn. ORB-SLAM [1] dùng một tỉ số điểm số; tôi *không* chép con số ngưỡng cụ thể của họ vào đây, vì ngưỡng ấy phụ thuộc vào thang phần dư, vào ngưỡng RANSAC, và vào loại đặc trưng — chép nó sang một hệ khác mà không hiểu là cách nhanh nhất để có một hằng số kỳ bí. Nguyên tắc thì chép được: nếu H khớp tốt hơn đáng kể thì cảnh phẳng hoặc xoay thuận, và trong cả hai trường hợp bạn chưa nên khởi tạo bằng E .

Bước bốn — kiểm tra và quyết định. Phân rã mô hình thẳng, triangulate, rồi kiểm kết quả trước khi chấp nhận. Đây là mục 4.

B. Vì sao không chỉ dùng thuật toán năm điểm

Đây là câu trả lời cho câu hỏi mở đầu thứ hai, và câu trả lời có ba tầng.

Tầng một: thuật toán năm điểm không suy biến với cảnh phẳng *theo nghĩa nó vẫn trả về một E xác định*. Nhưng nó không giải quyết được vấn đề thật — với cảnh phẳng, *bài toán* có nhiều nghiệm, không phải *thuật toán* có nhiều nghiệm. Thuật toán năm điểm chọn một nghiệm trong họ đó dựa trên năm điểm được lấy mẫu, và nghiệm ấy nhảy với nhiều theo cách không kiểm soát được.

Tầng hai, và đây là tầng quan trọng: với cảnh *gần* phẳng — một bức tường có vài vật nhô ra — bài toán không suy biến hoàn toàn nhưng có số điều kiện rất lớn. Đây là suy biến số học ở A/05 mục 2C, và đúng kiểu khó đoán nhất. Homography cho một cách *đo* mức độ phẳng: nếu H khớp gần như hoàn hảo thì cảnh gần phẳng, và ta biết mình đang ở vùng nguy hiểm.

Tầng ba, thực dụng nhất: khi H thẳng, ta *vẫn khởi tạo được* bằng cách phân rã H — nó cho $(R, \mathbf{t}/d, \mathbf{n})$, tức tư thế tương đối và pháp tuyến mặt phẳng, với tỉ lệ tương đối so với khoảng cách tới mặt phẳng. Đó là một khởi tạo hợp lệ cho cảnh phẳng, và nó tốt hơn nhiều so với việc ép E hoạt động. Nói cách khác, model selection không chỉ là một bộ phát hiện suy biến mà còn là một bộ chọn *thuật toán phù hợp*.

C. Tiêu chí chọn cặp khung

Đây là chỗ A/01 mục 6 trả công trực tiếp nhất.

Cần bao nhiêu parallax? Không có ngưỡng phổ quát, vì nó phụ thuộc vào σ của front-end (B/07 mục 4B) và vào độ sâu cảnh. Nhưng có một *cách tính* ngưỡng thay vì đoán nó, và cách ấy đáng biết.

Từ A/06, bất định độ sâu của một điểm triangulate với góc parallax α nhỏ tỉ lệ xấp xỉ $\sigma_z/z \approx \sigma_\theta/\alpha$, với $\sigma_\theta \approx \sigma_{px}/f$ là bất định góc của một quan sát. Đặt yêu cầu $\sigma_z/z \leq \epsilon$ — chẳng hạn $\epsilon = 0,1$ cho 10% sai số độ sâu tương đối — ta được

$$\alpha \geq \frac{\sigma_{px}}{f \epsilon}.$$

Với $\sigma_{px} = 0,2$, $f = 400$, $\epsilon = 0,1$: $\alpha \geq 0,005 \text{ rad} \approx 0,29^\circ$. (Tôi tự dẫn quan hệ này từ hình học tam giác nhỏ; chưa đối chiếu một nguồn cụ thể, và nó là xấp xỉ bậc nhất.)

Điều đáng nói không phải con số mà là *phương pháp*: ngưỡng parallax nên được *tính ra* từ σ đã đo và từ yêu cầu độ chính xác của ứng dụng, chứ không nên là một hằng số chép từ mã nguồn của người khác. Đây là một ví dụ nhỏ của nguyên tắc lớn hơn ở D/21: mỗi hằng số trong hệ thống nên truy được về một phép đo hoặc một yêu cầu.

Hai tiêu chí nữa. *Số lượng tương ứng*: cần đủ để RANSAC có ý nghĩa thống kê, và đủ để bản đồ ban đầu không quá nghèo. *Phân bố*: các điểm khởi tạo nên trải đều khung hình, vì lý do ở B/07 mục 4A — điều kiện số của bài toán khởi tạo cũng chịu ảnh hưởng như mọi bài toán khác.

I 1.3 Các trường hợp dễ hơn, và vì sao chúng dễ hơn

Điều đáng chú ý về mục này: mọi trường hợp dễ hơn đều dễ hơn vì *cùng một lý do*, và nhận ra điều đó là hiểu được cả chương.

A. Stereo và RGB-D

Với stereo đã hiệu chỉnh, khung đầu tiên đã cho một đám mây điểm có đơn vị mét: khớp dọc hàng, tính disparity, lấy $z = bf/d$. Không cần chuyển động, không cần parallax theo thời gian, không cần model selection. Hệ thống có bản đồ ngay ở khung một.

Vì sao dễ như vậy? Đây là câu trả lời cho câu hỏi mở đầu thứ ba. Theo A/05 mục 3B, đường cơ sở stereo là một *độ dài đã biết bằng mét*, nên nó phá vỡ gauge tỉ lệ ngay lập tức. Bảy bậc tự do của mono rút xuống sáu, và bậc bị mất chính là bậc khó nhất — bậc mà mono cần chuyển động để lấy. Nói khác đi: stereo mua thông tin bằng *phân cứng* thay vì bằng *chuyển động*.

Đây là một trong những lập luận mạnh nhất cho stereo trong sản phẩm, và nó lớn hơn lợi ích “có độ sâu trực tiếp” thường được nêu. Cả một lớp chế độ hồng — toàn bộ mục 2 của chương này, cộng bốn cấu hình suy biến ở A/01 — biến mất. Với robot phải khởi động ở một vị trí bất kỳ do khách hàng đặt, trong một môi trường mà ta không kiểm soát được, đó là khác biệt giữa dùng được và không.

Cái giá, đã nêu ở A/02 mục 5C: đường cơ sở phải giữ ổn định, và lợi ích tắt dần theo bình phương khoảng cách. Ngoài tầm hữu ích của stereo, ta trở lại chính bài toán mono — nên hệ stereo vẫn cần logic của mục 2, chỉ là ít khi phải dùng tới.

B. Thị giác-quán tính

Khởi tạo VI giải đồng thời cho hướng trọng lực, tỉ lệ metric, vận tốc ban đầu và độ lệch con quay, bằng cách căn chỉnh một quỹ đạo thị giác không có tỉ lệ với tiền tích phân IMU. VINS-Mono làm theo cách tuyến tính từng bước [2]; ORB-SLAM3 làm bằng một bài toán MAP thuần quán tính [3]. Chi tiết đầy đủ nằm ở `../IMU_Fusion/` chương 10 và không lặp lại ở đây.

Điều cần nhớ cho chương này là điều kiện: *gia tốc phải thay đổi*. Nếu gia tốc thật là hằng — đứng yên, hoặc chuyển động thẳng đều — thì thành phần gia tốc nhập hoàn toàn vào vectơ trọng lực chưa biết, và tỉ lệ không quan sát được dù có IMU. Đây là cùng nguyên lý ở A/05: cảm biến cho thông tin chỉ khi chuyển động kích thích nó.

Hệ quả thực hành đáng nhớ: một hệ VI khởi tạo trên một drone đang đậu trên bàn sẽ thất bại, và nó thất bại một cách *im lặng* nếu không có kiểm tra kích thích. Đây là ví dụ mở đầu của `../IMU_Fusion/` chương 5, và nó là lý do mọi bộ khởi tạo VI nghiêm túc đều có một cổng kiểm tra phương sai gia tốc.

C. Prior độ sâu học được

Một mô hình độ sâu metric đơn ảnh cho một ước lượng độ sâu ngay từ khung đầu, nên về nguyên tắc nó chơi vai trò của stereo. Điều này hữu ích và ngày càng được dùng.

Nhưng theo B/08 mục 4C, phải cẩn thận về cách đưa vào. Dùng đúng: *khởi tạo* tỉ lệ và độ sâu từ mô hình, rồi để hình học tiếp quản khi đã có đủ parallax. Dùng sai: giữ prior với trọng số cao mãi mãi, lúc đó một sai lệch có hệ thống của mô hình — và mọi mô hình đều có, phụ thuộc vào loại cảnh — sẽ chảy vào toàn bộ bản đồ.

Có một lợi ích thứ hai ít được nhắc và có thể quan trọng hơn: mô hình học được cho một ước lượng *ngay lập tức*, nên hệ thống có thể bắt đầu tracking trong khi chờ đủ parallax. Điều này biến giai đoạn bootstrapping từ “mù hoàn toàn” thành “ước lượng thô”, và với trải nghiệm người dùng thì đó là khác biệt lớn.

Bảng 1.1. Bốn cấu hình và cái mà mỗi cấu hình cung cấp. Cột thứ hai là cột giải thích tất cả: mỗi trường hợp dễ hơn vì có một nguồn phá vỡ gauge tỉ lệ.

Cấu hình	Nguồn phá gauge tỉ lệ	Điều kiện	Chế độ hỏng còn lại
Mono	Không có — tỉ lệ là gauge tự do	Cần parallax; cần cảnh không phẳng	Cả bốn cấu hình ở A/01 mục 6
Stereo	Đường cơ sở đã biết bằng mét	Không có — khung đầu là đủ	Ngoài tầm hữu ích thì thoái hoá về mono
RGB-D	Cảm biến độ sâu	Trong tầm và bề mặt hợp tác	Kính, gương, bề mặt đen, ngoài trời
Thị giác-quán tính	Gia tốc kế (qua tích phân)	Gia tốc phải thay đổi	Đứng yên hoặc thẳng đều thì thất bại im lặng
Prior học được	Tiên nghiệm về kích thước vật thể	Cảnh nằm trong phân bố huấn luyện	Sai lệch có hệ thống, không có bất định đã hiệu chuẩn

1.4 Tiêu chí từ chối: phần quan trọng nhất

A. Vì sao từ chối quan trọng hơn thành công

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất, và là luận điểm trung tâm của chương.

Một bộ khởi tạo có hai loại lỗi. *Từ chối khi lẽ ra chấp nhận được*: hệ thống chờ thêm vài khung. Chi phí là một chút độ trễ, và người dùng thấy hệ thống “hơi chậm khởi động”. *Chấp nhận khi lẽ ra phải từ chối*: hệ thống tạo ra một bản đồ sai, và mọi thứ sau đó xây trên bản đồ ấy. Chi phí là một quỹ đạo sai không tự sửa, và thường là mất bám hoàn toàn sau vài chục khung — nhưng đôi khi tệ hơn, là một quỹ đạo trông hợp lý mà sai tỉ lệ.

Hai chi phí ấy chênh nhau nhiều bậc độ lớn. Nên ngưỡng chấp nhận nên đặt *bảo thủ*, và một bộ khởi tạo từ chối thường xuyên là dấu hiệu của thiết kế đúng chứ không phải thiết kế kém. Một hệ chưa bao giờ từ chối thì hoặc nó chỉ chạy trong điều kiện lý tưởng, hoặc nó đang chấp nhận những thứ lẽ ra không nên — và bạn không biết là cái nào.

Nguyên tắc này lặp lại ở B/12 với relocalization, nơi chênh lệch còn lớn hơn, và ở B/11 với loop closure, nơi nó lớn nhất.

B. Bốn phép kiểm

Kiểm một — *parallax trung vị*. Không phải trung bình: trung vị bền hơn với vài điểm ở rất gần. Ngưỡng tính theo mục 2C từ σ đã đo. Đây là phép kiểm bắt được xoay thuần và chuyển động quá nhỏ.

Kiểm hai — *tỉ số điểm số giữa H và E*. Bắt được cảnh phẳng. Khi H thắng rõ rệt, hoặc chuyển sang khởi tạo bằng phân rã H, hoặc chờ thêm.

Kiểm ba — *tỉ lệ cheirality*. Sau khi phân rã và triangulate, đếm tỉ lệ điểm có độ sâu dương ở cả hai khung. Với nghiệm đúng, tỉ lệ này phải áp đảo. Nếu hai trong bốn nghiệm cho tỉ lệ xấp xỉ nhau thì đó *không* phải lúc chọn cái nhiều phiếu hơn — đó là dấu hiệu parallax quá nhỏ (A/01 mục 3E), và câu trả lời đúng là từ chối.

Kiểm bốn — *số điều kiện hoặc phổ giá trị riêng của Λ* . Sau khi dựng bài toán BA nhỏ cho hai khung, kiểm xem có bao nhiêu hướng gần suy biến ngoài bảy bậc gauge. Đắt hơn ba phép trên nhưng là phép kiểm tổng quát nhất — nó bắt được cả những trường hợp mà ba phép kia bỏ sót. A/05 khối *Cạm bẫy* liệt kê đầy đủ.

CẠM BÃY

Số inlier không nằm trong bốn phép kiểm, và đó là có chủ ý.

Như A/01 mục 6 đã lập luận, trong *cả bốn* cấu hình suy biến, số inlier vẫn cao. Nó đo mức nhất quán của dữ liệu với mô hình, không đo mức xác định của mô hình. Dùng nó làm tiêu chí chấp nhận là đúng loại nhầm lẫn mà A/05 mô tả.

Một biểu hiện cụ thể và rất hay gặp: hệ thống khởi tạo “thành công” với năm trăm inlier khi camera đang xoay tại chỗ trước một bức tường có vân. Mọi điểm khớp hoàn hảo — vì chúng thật sự khớp. Bản đồ tạo ra có mọi điểm ở những độ sâu ngẫu nhiên, và nó hoàn toàn nhất quán với dữ liệu.

Số inlier *vẫn* hữu ích, nhưng như một *điều kiện cần*, không phải điều kiện đủ: quá ít inlier thì chắc chắn không khởi tạo được, nhưng nhiều inlier không nói gì về việc có nên khởi tạo hay không.

C. Phát hiện khởi tạo hỏng sau khi đã chấp nhận

Bốn phép kiểm trên chạy *trước* khi chấp nhận. Cần thêm một tầng chạy *sau*, vì không bộ kiểm nào hoàn hảo.

Ba dấu hiệu cho thấy bản đồ ban đầu có vấn đề, quan sát được trong vài chục khung đầu. *Tỉ lệ tracking sụt nhanh*: nếu bản đồ ban đầu sai, các landmark không chiếu về đúng chỗ ở khung sau và bị loại dần. *Phần dư trung bình tăng dần* thay vì ổn định. *Nhiều landmark bị loại vì cheirality* — điểm rơi ra sau camera khi camera di chuyển, dấu hiệu độ sâu ban đầu sai.

Khi phát hiện, hành động đúng là *vứt bản đồ và khởi tạo lại* chứ không phải cố cứu. Đây là một quyết định kiến trúc quan trọng: hệ thống phải có khả năng vứt bỏ trạng thái của chính nó và bắt đầu lại sạch sẽ. Nhiều hệ không có khả năng đó và phải khởi động lại cả tiến trình — chấp nhận được trong nghiên cứu, không chấp nhận được trong sản phẩm (D/21).

ORB-SLAM3 đi xa hơn với kiến trúc Atlas: khi mất bám, nó bắt đầu một bản đồ *mới* và giữ bản đồ cũ; nếu sau này phát hiện hai bản đồ chồng lấn, nó ghép chúng [3]. Đây là cách xử lý tình tế hơn nhiều so với vứt đi, và nó là chủ đề của C/13.

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ORB-SLAM3 [3]	Model selection H so với E ; từ chối khi parallax không đủ; Atlas cho nhiều bản đồ	Cài đặt đầy đủ nhất của mục 2 và mục 4C
VINS-Mono [2]	SfM thuần thị giác rồi căn với IMU; kiểm phương sai gia tốc trước khi khởi tạo	Mục 3B — và công kiểm tra kích thích là một ví dụ tốt
DSO [4]	Khởi tạo direct coarse-to-fine với độ sâu ngẫu nhiên ban đầu	Cho thấy direct cần chiến lược khác hẳn: không có tương ứng rời rạc để dùng E hay H
SVO-Pro [5]	Khởi tạo bằng homography cho cảnh phẳng (giả định mặt đất)	Ví dụ về việc dùng tiên nghiệm ứng dụng — hợp lệ khi tiên nghiệm đúng
COLMAP [6]	Chọn cặp khung khởi tạo theo tiêu chí hình học, thử nhiều cặp	Ngoại tuyến nên có thể thử lại — một xa xỉ mà hệ thời gian thực không có

MINI PROJECT SLAM • Xây một bộ khởi tạo biết từ chối, rồi đo tỉ lệ từ chối đúng và sai • 3 ngày

Mục tiêu	biến “khởi tạo” từ một bước bạn hy vọng thành công thành một bước có tiêu chí đo được và có tỉ lệ lỗi biết trước.
Dữ liệu	một chuỗi EuRoC hoặc TUM-RGBD có chân trị, cộng <i>ba chuỗi tự quay</i> : một chuỗi xoay thuần, một chuỗi nhìn tường phẳng, một chuỗi đi thẳng về phía trước. Ba chuỗi sau là phần quan trọng nhất của bài này.
Các bước	cài khởi tạo mono hai khung với model selection; cài cả bốn phép kiểm ở mục 4B; chạy trên cả bốn chuỗi và ghi lại quyết định (chấp nhận/từ chối) kèm giá trị của bốn đại lượng kiểm; với các lần chấp nhận trên chuỗi có chân trị, đo sai số tỉ lệ và sai số tư thế tương đối; vẽ đường cong ROC — tỉ lệ chấp nhận sai so với tỉ lệ từ chối sai — khi quét ngưỡng parallax.
Xong khi	bạn có một đường cong ROC và một ngưỡng parallax chọn từ đường cong đó chứ không chép từ đâu. Bạn phải quan sát được rằng trên ba chuỗi suy biến, số inlier vẫn cao trong khi ba đại lượng kiểm kia thì không — xác nhận khối <i>Cạm bẫy</i> bằng dữ liệu của chính bạn. Và bạn có một con số: tỉ lệ chấp nhận sai của hệ thống bạn ở ngưỡng đã chọn.
Nối với	B/12-relocalization dùng cùng nguyên tắc từ chối với chênh lệch chi phí còn lớn hơn; D/21-tu-prototype-len-mass-production biến tỉ lệ chấp nhận sai thành một chỉ tiêu nghiệm thu.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì một bộ khởi tạo có hai loại lỗi với chi phí chênh nhau nhiều bậc. Từ chối sai chỉ tốn một chút độ trễ; chấp nhận sai tạo ra một bản đồ sai mà mọi thứ sau đó xây trên nó, và nó *không tự sửa*. Một hệ chưa bao giờ từ chối thì hoặc chỉ chạy trong điều kiện lý tưởng, hoặc đang chấp nhận những thứ lẽ ra không nên — và bạn không biết là cái nào, tức bạn đang bay mù. Thiết kế đúng là đặt ngưỡng bảo thủ và *đo* tỉ lệ chấp nhận sai, chứ không phải tối đa hoá tỉ lệ thành công.
- A2. Ba tầng. (a) Thuật toán năm điểm vẫn trả về một E xác định với cảnh phẳng, nhưng vấn đề là *bài toán* có nhiều nghiệm chứ không phải thuật toán — nghiệm nó chọn nhảy với nhiễu theo cách không kiểm soát được. (b) Với cảnh *gần* phẳng, bài toán có số điều kiện rất lớn — suy biến số học, khó chẩn đoán nhất — và homography cho một cách *đo* mức độ phẳng. (c) Khi H thẳng, ta vẫn khởi tạo được bằng phân rã H , vốn cho $(R, t/d, n)$ và là một khởi tạo hợp lệ cho cảnh phẳng. Nên model selection không chỉ phát hiện suy biến mà còn chọn thuật toán phù hợp.
- A3. Stereo cung cấp một *độ dài đã biết bằng mét* — đường cơ sở. Theo A/05 mục 3B, nó phá vỡ gauge tỉ lệ ngay lập tức, đưa bảy bậc tự do của mono xuống sáu, và bậc bị mất chính là bậc khó nhất — bậc mà mono phải dùng chuyển động để lấy. Nói khác đi: stereo mua thông tin bằng *phần cứng* thay vì bằng *chuyển động*. Hệ quả là cả một lớp chế độ hồng biến mất. Cái giá: đường cơ sở phải ổn định, và lợi ích tắt dần theo bình phương khoảng cách — ngoài tầm hữu ích thì ta trở lại đúng bài toán mono.
- A4. Không, BA *không* sửa được, và lý do là A/05: tỉ lệ là một gauge tự do của SLAM mono. BA tìm cấu hình nhất quán nhất với dữ liệu, và một bản đồ cơ giản đều với tỉ lệ sai là hoàn toàn nhất quán — mọi phần dư chiếu đều bằng đúng như với tỉ lệ đúng. Hàm mục tiêu phẳng theo hướng đó, nên BA không có tín hiệu nào để sửa. Muốn sửa thì phải *thêm thông tin*: một cảm biến có đơn vị mét, một vật có kích thước đã biết, hoặc một loop closure $Sim(3)$ nối về một phần bản đồ có tỉ lệ đúng.
- A5. Ba nguyên nhân: (a) *xoay thuận* — robot quay để vào hành lang, phát hiện bằng parallax trung vị thấp dù số inlier cao; (b) *cảnh phẳng* — camera nhìn vào tường cuối hành lang, phát hiện bằng tỉ số điểm số H thẳng E ; (c) *đi dọc trục quang* — robot đi thẳng dọc hành lang, epipole nằm trong ảnh, phát hiện bằng phân bố parallax rất lệch theo bán kính và bằng số điều kiện lớn. Ba đại lượng phân biệt chúng là ba đại lượng kiểm khác nhau ở mục 4B, và không cái nào là số inlier. Lưu ý thêm: hành lang thường có *cả ba* cùng lúc, và đó là lý do nó là môi trường khó nhất cho SLAM mono.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn bị bịt mắt đưa vào một căn phòng lạ, rồi tháo băng ra và bảo vẽ bản đồ căn phòng. Nhìn một lần không đủ. Một bức tranh vẽ cảnh sâu hun hút và một cái hành lang thật, từ một chỗ đứng, trông y hệt nhau. Bạn phải bước sang bên vài bước rồi nhìn lại: thứ gì dịch nhiều là gần, thứ gì gần như đứng im là xa.

Ba chuyện có thể làm hỏng việc này. Bạn *quay người* thay vì bước đi — cảnh đổi hoàn toàn nhưng chẳng học được gì. Bạn đang nhìn vào một *bức tường phẳng* — bước đi thì mọi thứ dịch, nhưng dịch giống hệt nhau nên chẳng đọc được gì từ sự khác biệt. Hoặc bạn *đi thẳng* về phía đang nhìn — mọi thứ to dần chứ không lệch sang bên, nên bạn chỉ có rất ít thông tin.

Điều mà một người tỉnh táo làm trong cả ba trường hợp: nói “khoan đã”, rồi bước sang bên vài bước nữa.

Điều mà một cái máy lập trình kém làm: vẽ ra một bản đồ. Vì người ta bảo nó phải vẽ ra một bản đồ, và nó không có khái niệm “chưa đủ dữ liệu”.

Và đây là chỗ nguy hiểm: bản đồ sai ấy không tự hỏng. Mọi thứ máy nhìn thấy về sau sẽ được đặt vào cho *khớp* với bản đồ sai ban đầu, nên bản đồ vẫn luôn tự nhất quán. Nó chỉ không giống căn phòng thật.

Nên phần khó nhất của chương này không phải cách vẽ bản đồ. Đó là cách dạy cái máy nhận ra lúc nó chưa nên vẽ — và chịu đựng việc phải chờ.

Chỗ không rút gọn được: chỗ không rút gọn được là *ngưỡng*: bước sang bên bao nhiêu thì đủ? Câu trả lời phụ thuộc vào camera của bạn chính xác tới đâu, cảnh xa hay gần, và bạn cần bản đồ chính xác

tới mức nào. Mục 2C cho một cách *tính* nó thay vì đoán, và đó là cách đúng — nhưng nó đòi bạn phải biết σ của chính mình, và đó là một phép đo mà ít ai làm.

Một câu: Một bộ khởi tạo tốt là bộ biết nói “chưa đủ dữ liệu” — vì cái nó tạo ra sẽ là hệ quy chiếu cho mọi thứ về sau.

1.5 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *khởi tạo sai không tự sửa được* — là hệ quả của A/05: tỉ lệ là gauge tự do nên BA không có tín hiệu để sửa nó. Nó bị bác bỏ nếu ai đó cho thấy một hệ mono thuần thị giác phục hồi được tỉ lệ đúng sau một khởi tạo sai tỉ lệ, mà không dùng thêm thông tin nào. Điều đó bất khả theo lý thuyết; điều *có thể* xảy ra và cần phân biệt là hệ phục hồi được sau một khởi tạo sai về *hình học* (không phải tỉ lệ) — chẳng hạn khi các landmark sai bị loại dần và thay bằng landmark mới đúng. Đó là một cơ chế thật và nó làm mệnh đề cần được phát biểu chặt hơn: *sai tỉ lệ không sửa được; sai hình học thì có thể, nếu hệ có cơ chế loại landmark tốt*.

Mệnh đề thứ hai — *số inlier không phải tiêu chí chấp nhận* — là mệnh đề kế thừa từ A/01 mục 6 và kiểm được bằng mini project (cửa b). Nó bị bác bỏ nếu trên một tập đủ lớn các lần khởi tạo gồm cả cấu hình suy biến, số inlier tương quan chặt với sai số tỉ lệ thật — chặt ngang hoặc hơn parallax trung vị.

Mệnh đề thứ ba — *ngưỡng parallax tính được từ σ và yêu cầu độ chính xác* — dựa trên một xấp xỉ bậc nhất mà tôi tự dẫn ở mục 2C và *chưa đối chiếu nguồn*. Nó có thể sai ở chỗ: quan hệ $\sigma_z/z \approx \sigma_\theta/\alpha$ bỏ qua ảnh hưởng của số lượng quan sát (nhiều quan sát bù được parallax nhỏ ở mức độ nào đó) và bỏ qua hình dạng phân bố lệch ở A/06. Một công thức đúng hơn nên tính từ ma trận thông tin đầy đủ. Đây là một mục trong 99-chua-biet.md.

Một mệnh đề mềm: chương này khẳng định *tỉ lệ từ chối cao là dấu hiệu thiết kế đúng*. Điều đó có thể sai trong ứng dụng mà độ trễ khởi động là yêu cầu cứng — một thiết bị AR mà người dùng không chờ quá hai giây. Ở đó, đánh đổi dịch chuyển, và cách đúng là dùng một cấu hình cảm biến khác (stereo, hoặc prior học được) chứ không phải nới ngưỡng. Nêu rõ điều này vì nguyên tắc ở mục 4A không phổ quát mà phụ thuộc vào tỉ số chi phí của hai loại lỗi.

1.6 Kết nối

Nối lên trên. A/01-hình-hoc-da-thi là chương mà chương này áp dụng trực tiếp nhất — mục 6 ở đó liệt kê bốn cấu hình suy biến, và mục 4B ở đây biến chúng thành bốn phép kiểm chạy được. A/05-observability-gauge-va-suy-bien cho nguyên tắc lớn hơn: tỉ lệ là gauge nên không sửa được, và điều kiện kích thích quyết định khi nào được phép tin. A/06-bat-dinh-va-lan-truyen-sai-so cho công cụ tính ngưỡng parallax từ σ . B/07-dac-trung-va-data-association cho σ đó.

Nối xuống dưới. B/11-loop-closure-va-nhat-quan-toan-cuc dùng cùng nguyên tắc từ chối với chênh lệch chi phí còn lớn hơn: một loop closure sai phá cả bản đồ. B/12-relocalization cũng vậy, và cả ba chương — 10, 11, 12 — có chung một cấu trúc lập luận: hai loại lỗi với chi phí bất đối xứng, nên ngưỡng phải bảo thủ và tỉ lệ lỗi phải được đo. C/13-quan-ly-ban-do nhận vấn đề ở mục 4C: hệ thống phải vứt bỏ được bản đồ của chính nó, và Atlas là cách xử lý tinh tế hơn.

Nối xa hơn. C/15-hop-nhat-da-cam-bien là chỗ Bảng 1.1 được đọc như một bảng thiết kế: chọn cảm biến nào chính là chọn phá gauge nào. D/19-ben-vung-trong-the-gioi-thuc bàn về việc khởi tạo trong điều kiện xấu — ít vân, ánh sáng kém, vật động.

D/21-tu-prototype-len-mass-production biến tỉ lệ chấp nhận sai thành một chỉ tiêu nghiệm thu, và lập luận rằng khởi động nhanh và đáng tin là một yêu cầu sản phẩm chứ không phải một tính năng.

Nối ngang: ../IMU_Fusion/ chương 10 là chương đầy đủ về khởi tạo thị giác-quán tính, với suy dẫn các bước giải cho trọng lực, tỉ lệ và vận tốc. Chương này chỉ nêu kết quả và điều kiện.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao một bộ khởi tạo chưa bao giờ từ chối là một tin xấu? Nêu tỉ số chi phí giữa hai loại lỗi.
2. Nêu ba lý do để chạy song song E và H thay vì chỉ dùng thuật toán năm điểm.
3. Tự dẫn lại ngưỡng parallax từ σ_{px} , f và yêu cầu ϵ , rồi tính cho $\sigma = 0,3$ px, $f = 600$, $\epsilon = 0,05$.
4. Điều gì stereo cung cấp mà mono không có? Trả lời bằng ngôn ngữ gauge, không bằng ngôn ngữ “có độ sâu”.
5. Khởi tạo VI cần gia tốc thay đổi. Vì sao, và chuyện gì xảy ra nếu bỏ qua điều kiện đó?
6. Nêu bốn phép kiểm trước khi chấp nhận khởi tạo, và nói mỗi phép bắt được cấu hình suy biến nào.
7. Bản đồ khởi tạo sai tỉ lệ 20%. Nêu chính xác vì sao BA không sửa được, và nêu ba cách thêm thông tin để sửa.

Nguồn

- [1] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM: A Versatile and Accurate Monocular SLAM System,” *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, 2015, mức tin A.
- [2] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018, mức tin A.
- [3] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.
- [4] J. Engel, V. Koltun, and D. Cremers, “Direct Sparse Odometry,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 3, pp. 611–625, 2018, mức tin A.
- [5] C. Forster, Z. Zhang, M. Gassner, et al., “SVO: Semidirect Visual Odometry for Monocular and Multicamera Systems,” *IEEE Transactions on Robotics*, vol. 33, no. 2, pp. 249–265, 2017, mức tin A.
- [6] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2016.